Table of Contents

RGB System Core Rules

English

RGB System

RGB System

1. RGB System — Documentation Index

This index organizes the complete RGB System documentation in English.

Its purpose is to provide a central navigation hub for all system modules.

2. Introduction

1.1 RGB System

The RGB System is a tabletop RPG designed around simple rules and tactical combat.

2.1 RGB System – Quick Start

This quick start guide explains the core ideas of the RGB System and how to begin playing in just a few minutes.

The system uses three core vectors:

```
Vector   Name     Function
------   -------  -------------------------------------R       Red      attacks, power, strength checks, accuracy, dam
G        Green    agility, mobility, evasion, reaction speed
B        Blue     energy shield, special protection, intellect, energy
```

```
| Vector | Name | Function |
|||--|
| R | Red | pressure, impact, disruption |
| G | Green | relation, movement, reaction |
| B | Blue | preservation, shields, stabilization |
```

7.1 Reference

This section contains reference materials and practical examples for the **RGB System**.

These documents help players and game masters understand how the rules work in practice.

7.1.1 Examples

Character Sheet

Standard character sheet template for RGB characters.

→ Character Sheet

Combat Example

Combat example using the system's rules.

→ Combat Example → Combat Walkthrough

Gameplay Example

Narrative example of the system in use during a session.

→ Gameplay Example

Gameplay Loop

Explains the typical flow of an RGB session.

→ Gameplay Loop

7.1.2 Design Notes

RGB Interaction Model

Explains the relationship between vectors, damage, and combat.

→ RGB Interaction Model

System Architecture

Observations on the RGB System's architecture.

→ RGB System Architecture

Extra Notes

Additional observations about the system's design.

→ Extra Notes

7.1.3 Known Gaps / Maintenance Notes

- **Link checking:** this documentation is currently checked manually. Adding an automated markdown link-checker to CI is recommended to prevent broken cross-references from going unnoticed.
- **RGB Ability Engine:** an earlier reference to a `rgb_ability_engine.md` document was removed from `rgb_system_engine.md` — no such document was ever authored in this repository.

← Back to README

7.2 Character Sheet

The character sheet summarizes the **core values of a character in the RGB System**.

It organizes the three RGB vectors together with derived values and equipment.

7.2.1 Basic Information

```
Name
Level
```

7.2.2 RGB Vectors

```
R (Red)   → Pressure / Impact
G (Green) → Relation / Reaction
B (Blue)  → Preservation / Energy
```

These vectors determine most actions in the system.

7.2.3 Derived Values

Derived values are calculated from RGB vectors.

```
Health = 4 + R + B
```

Optional energy shield (if the setting allows it):

```
Energy Shield = B × 3
```

Optional ability fatigue system:

```
Callback (if used in the campaign setting)
```

7.2.4 Equipment

```
Equipment
Weapons
Armor
```

Equipment interacts with the combat and damage systems.

7.2.5 Relationship with the RGB System

The character sheet connects several modules of the RGB system.

```
Vectors → define character capability
Health → derived from R + B
Shield → derived from B
Equipment → modifies combat interaction
```

7.2.6 Design Philosophy

The RGB character sheet follows three design principles:

- **minimal structure** – few numbers define the character
- **fast readability** – all core values visible at a glance
- **system modularity** – optional systems (shield, callback) can be added depending on the campaign

This allows the same sheet to work in:

- modern campaigns
- fantasy settings
- science-fiction worlds
- superhero games

7.2.7 Minimal Character Sheet Example

```
Name: Stormy
Level: 3

R: 3
G: 3
B: 1

Health: 8
Shield: 3

Armor: Medium Armor
Weapons: Dual Pistols
Equipment: Tactical Gear
```

← Back to README

7.3 Combat Example

This example demonstrates how combat is resolved using the RGB System.

Two characters engage in combat and resolve actions using the core rules:

- RGB vectors
- movement
- attack and defense
- armor and shields
- damage calculation

7.3.1 Characters

7.3.2 Character A – Assault Fighter

```
R = 3
G = 2
B = 1
```

Derived values:

```
Health = 4 + R + B = 8
Shield = B × 3 = 3
Movement = G × 2 = 4 meters
```

Equipment:

- Medium Armor (Protection 4)
- Rifle

7.3.3 Character B – Mobile Operator

```
R = 2
G = 3
B = 2
```

Derived values:

```
Health = 8
Shield = 6
Movement = 6 meters
```

Equipment:

- Light Armor (Protection 2)
- SMG

7.3.4 Combat Turn Example

7.3.5 Step 1 — Positioning

Characters move according to:

```
Movement = G × 2 meters
```

Character B moves first due to higher mobility.

7.3.6 Step 2 — Attack

Character A fires the rifle.

Example weapon damage:

```
Impact Source = rifle
Weapon Damage = 6
Penetration = 2
```

7.3.7 Step 3 — Armor Interaction

Armor reduces incoming damage after penetration.

Example:

```
Target Armor = 2
Penetration = 2
Effective Armor = 0
```

Armor is bypassed.

7.3.8 Step 4 — Shield Absorption

Shield absorbs damage before health.

```
Shield = 6
Damage = 6
```

Result:

```
Remaining Shield = 0
Health Damage = 0
```

7.3.9 Second Attack

Character B attacks with an SMG.

```
Weapon Damage = 4
Penetration = 1
```

Character A armor:

```
Armor = 4
Effective Armor = 3
```

Damage calculation:

```
Damage = 4 - 3 = 1
```

Character A receives:

```
Health Damage = 1
Remaining Health = 9
```

7.3.10 Damage Flow Summary

The RGB system resolves damage using a layered structure.

```
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

7.3.11 Tactical Interpretation

The example shows the role of the RGB vectors:

```
R → determines pressure and impact
G → determines relation, mobility, and positioning
B → determines preservation and shield capacity
```

Each vector supports a different combat strategy.

7.3.12 Design Goal

Combat resolution in the RGB system is designed to be:

- fast to calculate
- tactically meaningful
- consistent across settings

Because the same formulas apply to all characters, the system remains easy to balance.

7.3.13 See Also

- Attack and Defense
- Movement
- Damage Model

← Back to README

7.4 Combat Walkthrough

This example demonstrates a **complete combat encounter** using the RGB System.

Two characters engage in combat inside a tactical environment.

7.4.1 Characters

7.4.2 Stormy

R = 3
G = 3
B = 2

Derived values:

Health = 4 + R + B = 9
Shield = B × 3 = 6
Movement = G × 2 = 6 meters

Equipment:

- Medium Armor
- Dual Pistols

7.4.3 Security Agent

R = 2
G = 2
B = 1

Derived values:

Health = 8
Shield = 3
Movement = 4 meters

Equipment:

- Light Armor
- SMG

7.4.4 Initial Situation

The characters begin **6 meters apart**.

Stormy has partial cover behind a vehicle.

7.4.5 Turn 1

7.4.6 Movement

Stormy moves 2 meters to improve line of sight.

Movement formula:

Movement = G × 2

Stormy could move up to 6 meters, but chooses a smaller reposition.

7.4.7 Attack

Stormy fires one pistol.

Weapon damage:

Damage = 4
Penetration = 1

7.4.8 Armor Interaction

Target armor:

Light Armor = 2
Penetration = 1
Effective Armor = 1

Damage after armor:

4 − 1 = 3

7.4.9 Shield Interaction

Target shield:

Shield = 3

Shield absorbs all damage.

Remaining Shield = 0
Health Damage = 0

7.4.10 Turn 2

The security agent retaliates.

Weapon:

SMG Burst
Damage = 5
Penetration = 1

Stormy armor:

Medium Armor = 4
Effective Armor = 3

Damage:

5 − 3 = 2

Stormy shield absorbs damage.

Remaining Shield = 4
Health Damage = 0

7.4.11 Combat Summary

This walkthrough illustrates the RGB combat model.

Damage flow:

Weapon
↓
Penetration
↓
Armor
↓
Shield
↓
Character

7.4.12 Tactical Observation

Stormy benefits from:

- higher mobility (G)
- stronger shield (B)

The agent relies more on weapon damage.

This shows how **different RGB builds influence combat strategy**.

← Back to README

7.5 Mathematical Balance Observation in the RGB System

This document describes an **emergent property** of the RGB System that appears when the main system formulas are analyzed together.

Even though the system was not explicitly designed for this purpose, the RGB mechanics produce a **natural balance between the three vectors: R, G, and B**.

7.5.1 Core RGB Formulas

The core rules define the following relationships.

7.5.2 Health

```
Health = 4 + R + B
```

7.5.3 Movement

```
Movement = G × 2
```

7.5.4 Energy Shield

```
Shield = B × 3
```

7.5.5 Special Absorption

Special absorption allows characters to convert vector points into damage reduction.

```
R → physical impact pressure
B → energy or magical absorption
```

7.5.6 Vector Influence in Combat

Each RGB vector influences combat in a different way.

```
Vector   Combat Role
------   ----------------------
R        Pressure and physical impact
```

```
G        Mobility, evasion and positioning
B        Energy shield and special defenses
```

7.5.7 Resistance Scaling Example

Consider a character with **5 points in each vector**.

```
Health = 4 + 5 + 5 = 14
Shield = 5 × 3 = 15
```

Observation:

```
Shield ≈ Health
```

This indicates that **energy protection and physical resilience reach similar levels of durability**, but through different mechanics.

7.5.8 Balance Between R and B

Both vectors can provide survivability.

7.5.9 High R Character

Advantages:

- higher pressure and impact
- contribution to health through physical pressure tolerance

7.5.10 High B Character

Advantages:

- stronger energy shield
- protection against special damage

Result:

Both builds can reach similar levels of overall resistance using different systems.

7.5.11 Role of the G Vector

The Green vector does not increase raw durability.

Instead, it improves:

- evasion
- movement
- tactical positioning

This creates a third survival strategy:

```
Avoid damage instead of absorbing it
```

7.5.12 Vector Multipliers

```
Vector   Multiplier
------   ----------
R        +1 health and pressure
G        ×2 (movement)
B        ×3 (shield)
```

These multipliers create three defensive approaches:

```
R → pressure tolerance and impact
B → preservation through health contribution and shield
G → avoidance through mobility
```

7.5.13 Emergent Character Archetypes

Without defining explicit classes, the system naturally produces character styles.

```
Style            Dominant Vector
---------------  -------------Physical tank     High R
Evasive fighter  High G
Energy defender  High B
```

7.5.14 Hybrid Builds

Characters can also combine vectors effectively.

Example:

```
R = 6
B = 6
```

Derived values:

```
Health = 16
Shield = 18
```

Approximate total resistance:

```
16 + 18 = 34
```

This demonstrates that **hybrid builds remain competitive** within the RGB system.

7.5.15 The RGB Balance Triangle

The system can be visualized as a functional triangle.

```
        G
   evasion / mobility

R ---------------- B
health / absorption    shield / energy
```

Each vector represents a different strategy for survival.

7.5.16 Conclusion

The RGB System demonstrates a **structural balance between its three core vectors**.

- **R** increases direct resilience through health
- **B** increases external protection through shields
- **G** improves survival by avoiding damage

Together these mechanics create a naturally balanced system without requiring rigid classes or complex balancing rules.

← Back to README

7.6 Gameplay Example

This example illustrates how a typical session flows in the RGB System.

Gameplay usually alternates between three main phases:

- narrative interaction
- exploration
- combat

These phases form the core gameplay loop used in most RGB campaigns.

7.6.1 Example Scenario

A small team of operatives is investigating suspicious activity in an industrial district.

The group consists of:

- Stormy – mobile combat specialist
- Security Agent – trained tactical operator
- Support Technician – equipment and analysis specialist

The Game Master describes the environment and the players decide how to proceed.

7.6.2 Phase 1 — Narrative

The session begins with role-playing and narrative interaction.

Example:

The team receives information about illegal technology shipments in a warehouse district.

Players may:

- question NPCs
- gather information
- plan their approach
- negotiate or intimidate

No strict mechanical rules are required in this phase unless the Game Master requests tests.

7.6.3 Phase 2 — Exploration

Players move through the environment and interact with objects or locations.

Examples:

- searching buildings
- bypassing security systems
- scanning the environment
- avoiding patrols

The Game Master may request RGB checks when appropriate.

Example:

```
Agility Check → based on G
Strength Check → based on R
Technical / Energy interaction → based on B
```

Exploration encourages creative use of equipment and abilities.

7.6.4 Phase 3 — Combat

If a conflict occurs, the game enters the **combat phase**.

Combat uses the RGB combat system.

Basic sequence:

```
Movement
↓
Action (attack / ability)
↓
Defense resolution
↓
Damage calculation
```

Example encounter:

Stormy encounters two hostile agents inside the warehouse.

Players use:

- movement (G)
- attacks (R)
- shields or abilities (B)

The encounter is resolved using the RGB combat rules.

7.6.5 Session Resolution

After the encounter, the game returns to narrative play.

Possible outcomes:

- characters recover from combat
- players investigate the location
- new story elements emerge

This cycle continues throughout the session.

7.6.6 RGB Gameplay Loop

The overall gameplay structure can be summarized as:

```
Narrative
↓
Exploration
↓
Combat
↓
Recovery / Story Progression
```

This loop allows the RGB system to support different play styles while keeping rules simple.

7.6.7 Design Philosophy

The RGB system separates **narrative flow** from **combat mechanics**.

- narrative phases remain flexible
- exploration encourages player creativity
- combat uses clear tactical rules

This balance allows the same system to work in:

- modern campaigns
- fantasy worlds
- science-fiction settings
- superhero stories

7.6.8 See Also

- gameplay_loop.md
- combat_example.md

- combat_walkthrough.md

← Back to README

7.7 Gameplay Loop

The RGB System follows a simple gameplay loop that connects character creation, exploration, and combat.

This loop describes how the system is typically used during play.

7.7.1 1. Character Creation

Players create characters using the RGB vector system.

Characters distribute points between:

- **R (Red)** — power, attacks and physical strength
- **G (Green)** — agility, mobility and reaction
- **B (Blue)** — shields, energy and special resistance

Additional abilities and equipment may be selected depending on the campaign.

→ See Character Creation

7.7.2 2. Exploration and Interaction

During gameplay, characters interact with the environment using their RGB attributes.

Typical actions include:

- investigation
- movement
- social interaction
- environmental challenges

The game master determines how RGB attributes influence these actions.

7.7.3 3. Tactical Positioning

Before combat begins, characters position themselves in the environment.

The RGB System supports tactical combat using grid-based positioning.

Typical considerations include:

- distance between characters
- line of sight
- cover and terrain
- movement speed

→ See Movement

7.7.4 4. Combat Resolution

Combat is resolved using the RGB combat rules.

Combat interactions involve:

- attack and defense comparisons
- weapon damage
- armor and shield interaction
- tactical movement

→ See Attack and Defense

7.7.5 5. Damage and Consequences

Damage affects characters through:

- armor
- shields
- vitality

Special abilities may modify these interactions.

→ See Weapons

7.7.6 6. Recovery

After encounters, characters may recover depending on the rules defined by the game master.

Recovery may include:

- health recovery
- ability cooldowns
- equipment repair

7.7.7 Gameplay Cycle

A typical session follows this structure:

Character Creation
→ Exploration
→ Tactical Positioning
→ Combat

→ Consequences
→ Recovery

This cycle repeats as the story progresses.

← Back to README

7.8 RGB System – Damage Model and Interaction Design Notes

This document consolidates the design observations about the RGB System damage model, its mathematical balance, and how the RGB vectors interact with combat mechanics.

The goal of this document is to explain **why the system behaves consistently and remains balanced**, while keeping the rules simple.

7.8.1 RGB Interaction Model

The RGB System is built around three fundamental vectors:

- **R (Red)** — pressure, impact, disruption and physical force
- **G (Green)** — relation, mobility, timing and reaction
- **B (Blue)** — preservation, shields, energy and special resistance

These vectors interact to create tactical gameplay.

```
        G
   relation / reaction

R - B
pressure          preservation
```

Each vector influences a different aspect of gameplay:

Vector	Gameplay Role
R	Pressure, impact and physical force
G	Mobility, positioning and evasion
B	Preservation, energy shields and special defenses

7.8.2 System Interaction Flow

The RGB System connects character attributes, equipment and combat mechanics.

```
RGB Vectors
↓
Character Creation
↓
Equipment Selection
↓
Combat Interaction
↓
Damage Resolution
```

This modular structure allows the system to adapt to multiple settings such as:

- modern campaigns
- fantasy worlds
- science fiction settings
- super-powered universes

7.8.3 RGB Damage Model

Combat damage in the RGB System follows a layered structure.

```
Hit or Contact Check
↓
Impact Source
↓
Penetration
↓
Armor Reduction
↓
Shield Absorption
↓
Remaining Damage → Character
```

This layered approach ensures that weapons, armor and shields interact predictably.

7.8.4 Core System Formulas

The RGB system defines character durability using simple formulas.

```
Health = 4 + R + B
Shield = B × 3
```

Weapons, attributes, abilities or procedures declare the **impact source**, while armor provides **fixed reduction values**.

Penetration modifies the effective armor value.

7.8.5 Effective Armor

Penetration interacts with armor through a simple relationship:

```
Effective Armor = Armor − Penetration
```

Final damage becomes:

```
Damage After Armor = Impact Source − Effective Armor
```

This keeps calculations simple and predictable.

7.8.6 Natural Damage Scale

If the following ranges are used:

Element	Typical Scale
Weapons	3–10
Armor	1–6
Penetration	0–4

A natural balance relationship appears:

```
Impact Source ≈ Armor + Penetration
```

This ensures that weapons and defenses remain balanced.

7.8.7 Example

Light Weapon

```
Damage = 4
Penetration = 1
Armor = 4
```

```
Effective Armor = 4 − 1 = 3
Final Damage = 4 − 3 = 1
```

Heavy Weapon

```
Damage = 8
Penetration = 3
Armor = 6
```

```
Effective Armor = 6 − 3 = 3
Final Damage = 8 − 3 = 5
```

7.8.8 Shield Interaction

After armor reduction, shields absorb remaining damage.

```
Remaining Damage − Shield
```

Shields follow the RGB formula:

```
Shield = B × 3
```

This creates two defensive layers:

```
| Defense Type | Function |
|--|-|
Armor | Reduces damage per attack |
Shield | Absorbs accumulated damage |
```

This separation simplifies combat resolution.

7.8.9 Balance Rule of Thumb

When designing weapons and armor, the following approximation keeps combat balanced:

```
Armor ≈ Impact Source / 2
Penetration ≈ Impact Source / 3
```

This keeps most attacks meaningful while preserving defensive value.

7.8.10 Unexpected Advantage: Multiple Enemy Combat

One interesting property of the RGB damage model is that it performs well when characters face **multiple opponents simultaneously**.

Many RPG systems struggle with this scenario because defensive values scale poorly against repeated attacks.

In RGB, the layered model distributes damage naturally:

```
Impact Source → Penetration → Armor → Shield → Character
```

Armor

Armor reduces damage for each individual attack, preventing weak enemies from overwhelming armored characters too quickly.

Shield

Shields absorb total accumulated damage, acting as a buffer against multiple small attacks.

7.8.11 Tactical Consequences

This creates a stable combat dynamic.

```
| Enemy Type | Result |
||--|
Many weak enemies | Armor reduces most attacks |
Few strong enemies | Penetration becomes important |
Energy-heavy enemies | Shields become critical |
```

Because armor works per hit and shields work cumulatively, the system remains stable even with many attackers.

7.8.12 Design Conclusion

The RGB damage structure forms a clear defensive hierarchy:

```
Weapon
↓
Penetration
↓
Armor
↓
Shield
↓
Character
```

This structure:

- keeps calculations simple
- supports tactical combat
- scales naturally with equipment
- handles multi-enemy encounters well

The interaction between R, G and B vectors ensures that different character builds remain viable while encouraging tactical choices during combat.

← Back to README

7.9 RGB System Architecture – Layered Design Observation

This note documents an interesting structural property that appears when the RGB documentation is analyzed as a whole.

Although the RGB System was designed as a simple tabletop RPG system, the documentation naturally reveals a **layered architecture** similar to what is often found in game engines.

This structure improves clarity, modularity and long-term expandability.

7.9.1 Layered Architecture of the RGB System

When the documentation is organized by function, the system forms the following layers:

```
System Layer
↓
Gameplay Layer
↓
Combat Layer
↓
Damage Layer
↓
Equipment Layer
```

Each layer answers a different design question.

7.9.2 1. System Layer

Documents:

- System Overview
- RGB Damage Interaction Model

Purpose:

This layer explains the **core philosophy and structure of the system**.

It defines the RGB vectors:

- **R (Red)** – damage and physical power
- **G (Green)** – mobility and reaction
- **B (Blue)** – shields and energy defense

It also shows how the system modules connect.

Equivalent concept in game engines:

Core system design.

7.9.3 2. Gameplay Layer

Documents:

- Gameplay Loop
- Combat Decision Model

Purpose:

This layer explains **how players interact with the system during play**.

Typical flow:

``````text Character Creation ↓ Exploration ↓ Combat ↓ Recovery

```
During combat the player usually chooses between:

```text
Press
Reposition
Sustain
```

Which map naturally to:

```
Press → R
Reposition → G
Sustain → B
```

Equivalent concept in game engines:

Gameplay framework.

## 7.10 3. Combat Layer

Documents:

- Attack and Defense
- Movement

Purpose:

Defines how characters interact during encounters.

This includes:

- movement rules
- attack resolution
- defensive reactions
- positioning

Equivalent concept in game engines:

Combat system.

## 7.11 4. Damage Layer

Documents:

- Damage Model
- RGB Damage Interaction Model

Purpose:

Handles the internal logic of damage calculation.

The RGB damage engine follows this structure:

```
Weapon
↓
Penetration
↓
Armor
↓
Shield
↓
Character
```

This layer allows combat to remain tactical while keeping calculations simple.

Equivalent concept in game engines:

Damage engine.

## 7.12 5. Equipment Layer

Documents:

- Armor
- Shields
- Firearms
- Melee
- Explosives

Purpose:

Defines the **data values used by the combat and damage systems**.

Examples:

- weapon damage
- armor reduction
- penetration values
- shield capacity

Equivalent concept in game engines:

Data layer.

## 7.13 Complete RGB System Architecture

When combined, the RGB documentation forms the following structure:

```
System Design
   |
   ├─ System Overview
   ├─ RGB Interaction Model
   |
Gameplay
   |
   ├─ Gameplay Loop
   ├─ Combat Decision Model
   |
Combat
   |
   ├─ Movement
   ├─ Attack & Defense
   |
Damage Engine
   |
   ├─ Damage Model
   |
```

```
Equipment Data
   |
   ├─ Armor
   ├─ Shields
   ├─ Weapons
```

## 7.14 Design Consequences

This layered architecture produces several advantages.

### Modularity

New subsystems can be added without modifying core rules.

Examples:

- magic systems
- cybernetics
- vehicles
- superpowers

### Easier Balance

Most balancing occurs in:

```
Damage Layer
Equipment Layer
```

The rest of the system remains stable.

### Clear Documentation

Each layer answers a different question:

```
| Layer | Question |
||-|
System | How the system is structured |
Gameplay | How players interact with the game |
Combat | How characters interact |
Damage | How damage is calculated |
Equipment | Which numerical values exist |
```

## 7.15 Final Observation

The RGB System behaves less like a traditional RPG rule set and more like a **reusable RPG system engine**.

This makes the system particularly well suited for:

- modern campaigns
- fantasy worlds
- science fiction settings
- super-powered universes

In this model:

```
RGB System → rule engine
Setting (e.g., Aurora) → world built on top of the engine
```

This separation allows the RGB System to remain generic while different settings provide narrative context.

← Back to README

# 7.16 RGB System Engine

The **RGB System Engine** describes the full architecture of the RGB role-playing system. It connects all system components into a layered structure similar to a game engine.

This model helps explain how the RGB system remains:

- modular
- scalable
- easy to balance
- adaptable to many settings

## 7.16.1 RGB System Architecture

The RGB system can be understood as a layered engine:

```
          Campaign Setting
                ↓
       RGB Ability Engine
                ↓
         Gameplay Layer
                ↓
          Combat System
                ↓
           Damage Engine
                ↓
          Equipment Data
                ↓
           RGB Vectors
```

Each layer builds on the one below it.

## 7.16.2 Layer Description

## 7.16.3 RGB Vectors (Core Layer)

The entire system is built on the three RGB vectors.

```
      G
mobility / positioning
```

```
R ---------------- B
power / damage      shield / defense
```

Vectors define the core mechanics:

- **R (Red)** → offensive power
- **G (Green)** → mobility and reaction
- **B (Blue)** → defense and energy

All other systems derive from these three values.

## 7.16.4 Equipment Data

This layer defines numerical values used by the combat system.

Examples:

- weapon damage
- armor reduction
- penetration values
- shield capacity

Documents:

- Firearms
- Melee
- Armor
- Shields

## 7.16.5 Damage Engine

The damage engine determines how attacks affect characters.

```
Weapon
↓
Penetration
↓
Armor
↓
Shield
↓
Character
```

Documents:

- Damage Model
- RGB Damage Interaction Model

## 7.16.6 Combat System

The combat layer defines interaction between characters.

Examples:

- attack resolution
- movement
- positioning
- defense reactions

Documents:

- Movement
- Attack and Defense

## 7.16.7 Gameplay Layer

Defines how players interact with the system during play.

Typical gameplay loop:

```
Character Creation
↓
Exploration
↓
Combat
↓
Recovery
```

Documents:

- Character Creation
- Gameplay Loop
- Combat Decision Model

## 7.16.8 RGB Ability Engine

This layer introduces abilities, powers, and special mechanics.

Abilities are built using the RGB vectors.

```
Vector
Cost
Effect
Duration
Limit
```

Documents:

- Skills and Abilities

## 7.16.9 Campaign Setting

The top layer defines the narrative world.

Examples:

- modern campaigns
- fantasy worlds
- science fiction
- superhero universes

Example:

```
RGB System → rule engine
Aurora → campaign setting built on top of the engine
```

## 7.16.10 Complete System Flow

The full interaction between layers:

```
Vectors
 ↓
Equipment Data
 ↓
Damage Engine
 ↓
Combat System
 ↓
Gameplay Rules
 ↓
Ability Engine
```

```
  ↓
Campaign Setting
```

## 7.16.11 Design Advantages

The RGB System Engine provides several advantages.

## 7.16.12 Modular Design

New subsystems can be added without rewriting core rules.

Examples:

- magic systems
- vehicles
- cybernetics
- superpowers

## 7.16.13 Easier Balance

Balance occurs primarily in:

```
Damage Engine
Equipment Data
```

The rest of the system remains stable.

## 7.16.14 Clear Documentation

Each layer answers a specific question.

```
Layer            Question
---------------------------------------------Vectors          What defines a character?
Equipment        What numbers exist?
Damage Engine    How damage works?
Combat System    How characters interact?
Gameplay         How players interact?
Abilities        What special powers exist?
Setting          What world the story happens in?
```

## 7.16.15 Final Observation

The RGB System behaves less like a traditional RPG ruleset and more like a **modular RPG engine**.

Because of this architecture, the same core system can support many different genres while remaining simple and consistent.

← Back to README